SQL

Basi

- Ci focalizziamo sulla sintassi SQL di base
- Applicabile a tutti i motori SQL (MS SQL Server, MySQL, PostgreSQL…)

- Analizziamo la sintassi per costruire una query SQL

- **Query** : A request for information from the database.

SELECT

- **SELECT** è l'istruzione più comunemente utilizzata e ci consente di recuperare informazioni da una tabella

SELECT - Sintassi

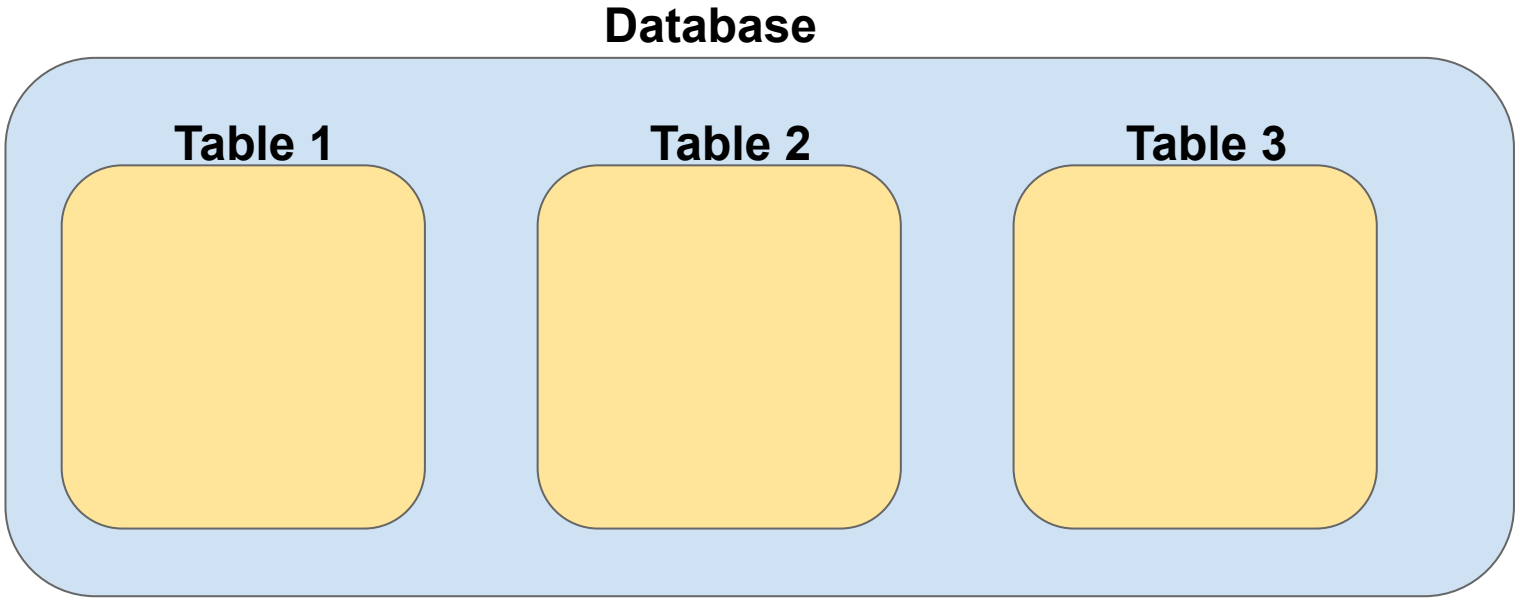
```
SELECT column_name  FROM table_name
```

SELECT column_name **FROM** table_name

Database

SQL

SELECT column_name **FROM** table_name



SQL

SELECT column_name **FROM** table_name

Database

Table 1

c1	c2	c3
x	23	a
y	18	b
z	46	c

Table 2

c1	c2	c3
1	Q	13
2	R	34
3	S	56

Table 3

c1	c2	c3
c	0	12
b	0	24
b	1	45

SQL

SELECT column_name **FROM** table_name

Database

Table 1

c1	c2	c3
x	23	a
y	18	b
z	46	c

Table 2

c1	c2	c3
1	Q	13
2	R	34
3	S	56

Table 3

c1	c2	c3
c	0	12
b	0	24
b	1	45

SQL

SELECT column_name **FROM** table_name

Database

Table 1

c1	c2	c3
x	23	a
y	18	b
z	46	c

Table 2

c1	c2	c3
1	Q	13
2	R	34
3	S	56

Table 3

c1	c2	c3
c	0	12
b	0	24
b	1	45

SQL

SELECT c1 **FROM** table_1

Database

Table 1

c1	c2	c3
x	23	a
y	18	b
z	46	c

Table 2

c1	c2	c3
1	Q	13
2	R	34
3	S	56

Table 3

c1	c2	c3
c	0	12
b	0	24
b	1	45

SQL

SELECT c1, c2 **FROM** table_1

Database

Table 1

c1	c2	c3
x	23	a
y	18	b
z	46	c

Table 2

c1	c2	c3
1	Q	13
2	R	34
3	S	56

Table 3

c1	c2	c3
c	0	12
b	0	24
b	1	45

SQL

SELECT c1, c3 **FROM** table_1

Database

table_1

c1	c2	c3
x	23	a
y	18	b
z	46	c

Table 2

c1	c2	c3
1	Q	13
2	R	34
3	S	56

Table 3

c1	c2	c3
c	0	12
b	0	24
b	1	45

SQL

SELECT * **FROM** table_1

Database

table_1

c1	c2	c3
x	23	a
y	18	b
z	46	c

Table 2

c1	c2	c3
1	Q	13
2	R	34
3	S	56

Table 3

c1	c2	c3
c	0	12
b	0	24
b	1	45

- In generale, non è buona norma utilizzare un asterisco (*) nell'istruzione SELECT se non sono realmente necessarie tutte le colonne.

- Interroga automaticamente tutto, il che aumenta il traffico tra il server del database e l'applicazione, il che può rallentare il recupero dei risultati.

SELECT

CHALLENGE TASKS

SQL

- Scenario
  - Vogliamo inviare un’email promozionale ai nostri clienti

SQL

- Richiesta
  - Utilizza un'istruzione **SELECT** per acquisire il nome e il cognome di ogni cliente e il relativo indirizzo e-mail.

SQL

- Risposta attesa: (non consideriamo l’ordine)

Data Output | Explain | Messages | Notifications

	first_name character varying (45)	last_name character varying (45)	email character varying (50)
1	Jared	Ely	jared.ely@sakilacustomer.org
2	Mary	Smith	mary.smith@sakilacustomer...
3	Patricia	Johnson	patricia.johnson@sakilacust...
4	Linda	Williams	linda.williams@sakilacusto...
5	Barbara	Jones	barbara.jones@sakilacusto...
6	Elizabeth	Brown	elizabeth.brown@sakilacust...

SQL

- Suggerimenti:
  - Usa la tabella **customer**
  - Verifica quali colonne sono definite nella tabella **customer**
  - Puoi usare **`SELECT * FROM customer`** per visualizzare tutte le colonne

DISTINCT

- A volte una tabella contiene una colonna con valori duplicati e potresti trovarti in una situazione in cui desideri solo elencare i valori univoci/distinti.
- La parola chiave **DISTINCT** può essere utilizzata per restituire solo i valori distinti in una colonna.

- La parola chiave **DISTINCT** opera su una colonna. La sintassi è simile a questa:

```
SELECT DISTINCT column FROM table
```

- Per chiarire a quale colonna viene applicato DISTINCT, puoi anche utilizzare le parentesi per chiarezza:

```
SELECT DISTINCT(column) FROM table
```

SQL

- Cosa significa quindi utilizzare DISTINCT su una colonna?

Name	Choice
Zach	Green
David	Green
Claire	Yellow
David	Red

SQL

- Immagina una tabella di persone interrogate sul proprio colore preferito

Name	Choice
Zach	Green
David	Green
Claire	Yellow
David	Red

SQL

- SELECT DISTINCT name FROM color_table

Name	Choice
Zach	Green
David	Green
Claire	Yellow
David	Red

SQL

- SELECT DISTINCT name FROM color_table

Name
Zach
David
Claire

- Dato l'esempio precedente, non sappiamo davvero se la persona con il nome "David" fosse una voce duplicata o due persone diverse con lo stesso nome.
- L’uso di DISTINCT ha risposto alla domanda: *quali sono i nomi univoci presenti nella tabella?*

- Ha più senso chiedere: *quanti tipi di scelte cromatiche uniche c'erano?*

SQL

- SELECT DISTINCT choice FROM color_table

Choice
Green
Yellow
Red

SELECT DISTINCT

CHALLENGE TASKS

SQL

- Scenario
  - Un visitatore italiano non conosce il sistema di valutazione americano per i film MPAA (e.g. PG , PG-13, R, etc...)
  - Vogliamo sapere le categoria presenti nel nostro database
  - Quali categorie sono disponibili?

SQL

- Risultato atteso
  - Data Output | Explain

	rating mpaa_rating
1	NC-17
2	G
3	PG
4	PG-13
5	R

COUNT

- La funzione COUNT restituisce il numero di righe di input che corrispondono a una condizione specifica di una query.
- Possiamo applicare COUNT su una colonna specifica o semplicemente passare COUNT(*) , vedremo presto che questo dovrebbe restituire lo stesso risultato.

SQL

Name	Choice
Zach	Green
David	Green
Claire	Yellow
David	Red

SQL

- SELECT COUNT(name) FROM table;

Name	Choice
Zach	Green
David	Green
Claire	Yellow
David	Red

SQL

- SELECT COUNT(name) FROM table;

Count
4

SQL

- SELECT COUNT(name) FROM table;
- Ritorna il numero di righe nella tabella
- È indipendente dalle colonne

Count
4

SQL

- Ogni colonna ha lo stesso numero di righe

Name	Choice
Zach	Green
David	Green
Claire	Yellow
David	Red

SQL

- SELECT COUNT(name) FROM table;
- SELECT COUNT(choice) FROM table;
- SELECT COUNT(*) FROM table;
- Restituiscono tutte lo stesso risultato

Count
4

- Il COUNT può essere utilizzato combinato ad altri comandi, come il DISTINCT

SQL

- *Quanti nomi univoci ci sono nella tabella?*

Name	Choice
Zach	Green
David	Green
Claire	Yellow
David	Red

SQL

- SELECT COUNT(DISTINCT name) FROM table;

Name	Choice
Zach	Green
David	Green
Claire	Yellow
David	Red

SQL

- SELECT COUNT(DISTINCT name) FROM table;

Name	Choice
Zach	Green
David	Green
Claire	Yellow
David	Red

SQL

- SELECT COUNT(DISTINCT name) FROM table;

Name	Choice
Zach	Green
David	Green
Claire	Yellow

SQL

- SELECT COUNT(DISTINCT name) FROM table;

Count
3

SQL

- SELECT COUNT(DISTINCT name) FROM table;

Count
3

SQL

- SELECT COUNT(DISTINCT(name)) FROM table;

Count
3

SELECT WHERE

- WHERE consente di definire delle condizioni sulle righe che devono essere restituite
- Sintassi di base

```
SELECT column1, column2
FROM table
WHERE conditions;
```

- La clausola WHERE appare immediatamente dopo la clausola FROM dell'istruzione SELECT.
- Le condizioni vengono utilizzate per filtrare le righe restituite dall'istruzione SELECT.

Operatori di confronto

Operator	Description
=	Equal
>	Greater than
<	Less Than
>=	Greater than or equal to
<=	Less than or equal to
<> or !=	Not equal to

Operatori logici

- AND
- OR
- NOT

SQL

- **SELECT** name,choice **FROM** table

Name	Choice
Zach	Green
David	Green
Claire	Yellow
David	Red

SQL

- **SELECT** name,choice **FROM** table

Name	Choice
Zach	Green
David	Green
Claire	Yellow
David	Red

SQL

- **SELECT** name,choice **FROM** table **WHERE** name = ‘David’

Name	Choice
David	Green
David	Red

SQL

- **SELECT** name **FROM** table **WHERE** name = ‘David’

Name
David
David

SQL

- **SELECT** name,choice **FROM** table **WHERE** name = ‘David’

Name	Choice
David	Green
David	Red

SQL

- **SELECT** name,choice **FROM** table **WHERE** name = ‘David’ **AND** choice= ‘Red’

Name	Choice
David	Red

SELECT WHERE

CHALLENGE TASKS

Scenario

- Un cliente ha dimenticato il portafoglio in uno dei nostri punti vendita. Dobbiamo rintracciare la sua email per informarli.
- Qual è l'e-mail per il cliente con il nome Nancy Thomas?

Risultato atteso

	email character varying (50)
1	nancy.thomas@sakilacustomer.org

Quesito 2

Scenario

- Un cliente vuole sapere di cosa parla il film "Outlaw Hanky". Potresti dare loro la descrizione del film "Outlaw Hanky"?

Quesito 3

- Un cliente è in ritardo con la restituzione del film e gli abbiamo inviato una lettera al suo indirizzo "259 Ipoh Drive". Dovremmo anche contattarlo al telefono per farglielo sapere.
- Cerchiamo il numero di telefono del cliente che vive al "259 Ipoh Drive"?

ORDER BY

- È possibile utilizzare ORDER BY per ordinare le righe in base a un valore di colonna, in ordine crescente o decrescente.
- Sintassi base:

```
SELECT column_1,column_2
FROM table
ORDER BY column_1 ASC / DESC
```

- Nota ORDER BY verso la fine di una query:
  - prima eseguiamo qualsiasi selezione e filtraggio,
  - poi ordiniamo i risultati
- Utilizzare ASC per ordinare in ordine crescente
- Utilizzare DESC per ordinare in ordine decrescente
- ORDER BY utilizza ASC per impostazione predefinita.

- È possibile usare ORDER BY su più colonne
- Questo ha senso quando una colonna ha voci duplicate.

Company	Name	Sales
Apple	Andrew	100
Google	David	500
Apple	Zach	300
Google	Claire	200
Xerox	Steven	100

- **SELECT** company,name,sales **FROM** table **ORDER BY** company,sales

Company	Name	Sales
Apple	Andrew	100
Apple	Zach	300
Google	Claire	200
Google	David	500
Xerox	Steven	100

ORDER BY

CHALLENGE TASKS

Quesito 1

- Vogliamo premiare i nostri primi 10 clienti paganti.
- Quali sono gli ID cliente dei primi 10 clienti che hanno fatto un pagamento?

Quesito 2

- Un cliente desidera noleggiare rapidamente un video da guardare durante la breve pausa pranzo.
- Quali sono i titoli dei 5 film più brevi (in termini di durata)?

Quesito 3

- Se il cliente precedente può guardare qualsiasi film della durata massima di 50 minuti, quante opzioni ha a disposizione?

BETWEEN

- L'operatore BETWEEN può essere utilizzato per confrontare un valore con un intervallo di valori:
-

```
value BETWEEN low AND high
```

- L’operatore BETWEEN equivale a:

```
value >= low AND value <= high
```

- Si può anche combinare BETWEEN con l'operatore logico NOT:

```
value NOT BETWEEN low AND high
```

- L'operatore NOT BETWEEN è uguale a:

```
value < low OR value > high
```

- L'operatore BETWEEN può essere utilizzato anche con le date. Tieni presente che devi formattare le date nel formato standard ISO 8601, ovvero AAAA-MM-GG

```
date BETWEEN ‘2007-01-01’  AND ‘2007-02-01’
```

IN

- In alcuni casi si desidera verificare la presenza di più possibili opzioni di valore, ad esempio se il nome di un utente viene visualizzato IN un elenco di nomi noti.
- Possiamo utilizzare l'operatore IN per creare una condizione che controlla se un valore è incluso in un elenco di più opzioni.

Sintassi generale:

```
value IN (option1,option2,...,option_n)
```

Esempi

```
SELECT color FROM table

WHERE color IN (‘red’,’blue’)


SELECT color FROM table

WHERE color NOT IN (‘red’,’blue’)
```

LIKE

- Siamo già stati in grado di eseguire confronti diretti con le stringhe, ad esempio:
- WHERE first_name= ‘John’
- Ma cosa succede se vogliamo confrontare uno schema generale in una stringa?
- Tutte le email che terminano con "@gmail.com"
- Tutti i nomi che iniziano con una "A"

- L'operatore LIKE ci consente di eseguire la corrispondenza del modello rispetto ai dati della stringa con l'uso di caratteri jolly:
- Percentuale %
  - Corrisponde a qualsiasi sequenza di caratteri
- Sottolineato _
  - Corrisponde a qualsiasi singolo carattere

- Tutti i nomi che iniziano con una "A"
  - WHERE nome LIKE 'A%'
- Tutti i nomi che terminano con una "a"
  - WHERE nome LIKE '%a'

- L'uso del carattere di sottolineatura ci consente di sostituire solo un singolo carattere
- Ottieni tutti i film di Mission Impossible

```
WHERE title LIKE ‘Mission Impossible _’
```

Puoi utilizzare più caratteri di sottolineatura

Immagina di avere i codici delle stringhe di versione nel formato "Versione#A4", "Versione#B7", ecc...

WHERE valore LIKE 'Versione#__'

- Puoi utilizzare più caratteri di sottolineatura
- Immagina di avere i codici delle stringhe di versione nel formato "Versione#A4", "Versione#B7", ecc...

```
WHERE valore LIKE 'Versione#_ _'
```

Possiamo anche combinare operatori di pattern matching per creare pattern più complessi

WHERE nome LIKE '_her%'

- Cheryl
- Theresa
- Sherri